Case Study — Securing a Legal Firm's Web Platform (Firebase / GCP)

Role: Sole technical owner (self-taught) · **Client:** Colombian law firm · **Stack:** Firebase (Hosting, Auth/Identity Platform, Firestore, Storage, Cloud Functions), GCP, App Check.

> *Versión en español:* `../caso-de-estudio-devsecops.md`
>
> A production case study documenting how I took a functional-but-insecure legal website to a hardened, monitored, and compliant platform — implementing authentication, RBAC, a pentest with remediation, backups, monitoring, and data-protection compliance.

Context & problem

The firm had a working website but with no real security posture: weak admin authentication, no access control by role, a third-party contact form, no backups, no monitoring, and no data-protection compliance. As the sole technical owner — learning cloud security along the way — I was responsible for closing those gaps end to end.

Objective

Reach a **defense-in-depth** posture (prevention / detection / recovery) covering the full **AAA** model — Authentication, Authorization, Accounting — without adding operational cost or third-party attack surface.

What I implemented

1. Authentication (MFA/TOTP) — blocker resolved

Enabled TOTP multi-factor auth on the admin panel. **Key debugging win:** the enrollment flow failed because `auth.currentUser` was `null` at the moment of enrollment; root-caused the session-timing issue and restored the correct flow. *(Lesson: MFA enrollment must happen against a fully-resolved authenticated session.)*

2. Authorization (RBAC)

Role model via **custom claims**: `admin` (full), `abogado` (content, inquiries, legal cases), `editor` (blog only). Enforced with granular **Firestore & Storage security rules** and mirrored in the UI (sections hidden per role — defense in depth, not UI-only).

3. Secure contact form

Replaced a third-party form service with a **Cloud Function** (`<enviar-consulta>`) that validates server-side, enforces **App Check** , stores inquiries with the Admin SDK, and sends the notification from a dedicated account

— **removing a third party from the trust boundary**.

4. Legal-cases module + private documents

CRUD for insolvency cases with document upload to **private Storage** (no public tokenized URLs; authenticated `getBlob` downloads), audit fields, and size/type limits in Storage rules.

5. Accounting (audit logs)

Append-only `auditoria` collection (create for admin/abogado, read admin-only) recording sensitive actions server-side — including an **admin access log**.

6. Authorized penetration test + remediation

Ran a 3-phase pentest (recon, authz testing, privilege-escalation attempts). - **Critical finding:** the `.git` directory (and `functions/package.json`) were being served by Hosting, leaking source code **and a GitHub token**. - **Fix:** corrected `firebase.json` (`ignore` rules + security headers: X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, CSP `frame-ancestors`). - RBAC / App Check / Storage were validated live (as anonymous and as `editor`): **no privilege escalation** — all controls held.

7. Recovery (backups, 3-2-1)

Firestore **PITR (7d)** + daily scheduled backups (14d) + Storage object versioning, plus an **offsite** weekly export to a separate-region bucket (Cloud Scheduler + dedicated service account, lifecycle 90d).

8. Detection & monitoring

- **File integrity monitoring:** cryptographic hashes of the live site files, compared automatically every 6 hours, alerting on any change that wasn't deployed.
- **Log-based alerting** in Cloud Monitoring (`severity>=ERROR`) → email + SMS.
- Supply chain: **Dependabot** + `npm audit` gate in CI; upgraded dependencies from 20 vulnerabilities (1 critical / 5 high) to 8 moderate.

9. Secrets management

Moved external service credentials out of a project file and into a **managed encrypted store**. A config file with credentials is still a file on disk: a backup copies it, a careless `git add` publishes it, anyone with access to the machine reads it.

10. Compliance (Law 1581 — Colombia's data-protection law)

Explicit consent capture with stored proof, an international-transfer clause in the privacy policy, and a documented roadmap for the remaining organizational items (Google DPA, national database registry, corporate identifiers).

Threat modeling & documentation

Produced a **PASTA** threat model (stages 3–7: decomposition, threat analysis, CWE/OWASP vulnerabilities, attack trees with validated controls, residual-risk matrix), **NIST incident-response playbooks**, and a **security-awareness training** program mapped to **CIS Control 14 / NIST CSF PR.AT**.

Results

- Full **AAA** coverage: Authentication (MFA), Authorization (RBAC), Accounting (audit logs).
- Defense in depth across **prevention / detection / recovery**.
- A **critical secret-exposure** vulnerability found and remediated before exploitation.
- Zero added third-party attack surface; no added recurring cost.
- A complete, portfolio-grade security documentation set (pentest report, backup runbook, IR playbooks, training, threat model).

Skills demonstrated

Cloud security (GCP/Firebase) · IAM & RBAC · security rules as code · penetration testing & remediation · secrets management · backup/DR (3-2-1, RPO/RTO) · FIM & log-based detection · supply-chain security (Dependabot, `npm audit`) · threat modeling (PASTA) · incident response (NIST) · data-protection compliance.
